



Database Integration

Lecture

Prof. Dr. Michael Kaufmann

Hochschule Luzern – Informatik

February 26, 2024

Agenda

- 1.) SQL-DML
- 2.) Real World Data Sources
- 3.) ETL
- 4.) EL-T

1. SQL-DML

Data Manipulation Language

Information Technology

October 7, 2025

FH Zentralschweiz

CREATE TABLE: Defining tables

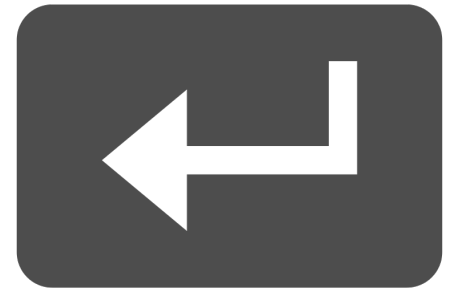
- SQL has a `CREATE TABLE` command for defining a table. The Employee table is specified as follows:

```
CREATE TABLE Employee
(   E#      CHAR(6) NOT NULL,
    Name  VARCHAR(20),
    ... )
```



- The opposite `DROP TABLE` command lets you delete table definitions.
- The `ALTER TABLE` command lets you change table definitions.
- With `CREATE TABLE AS SELECT ...` a table can be created as the output of a query. The table definition is automatically generated this way.

INSERT: Insert data



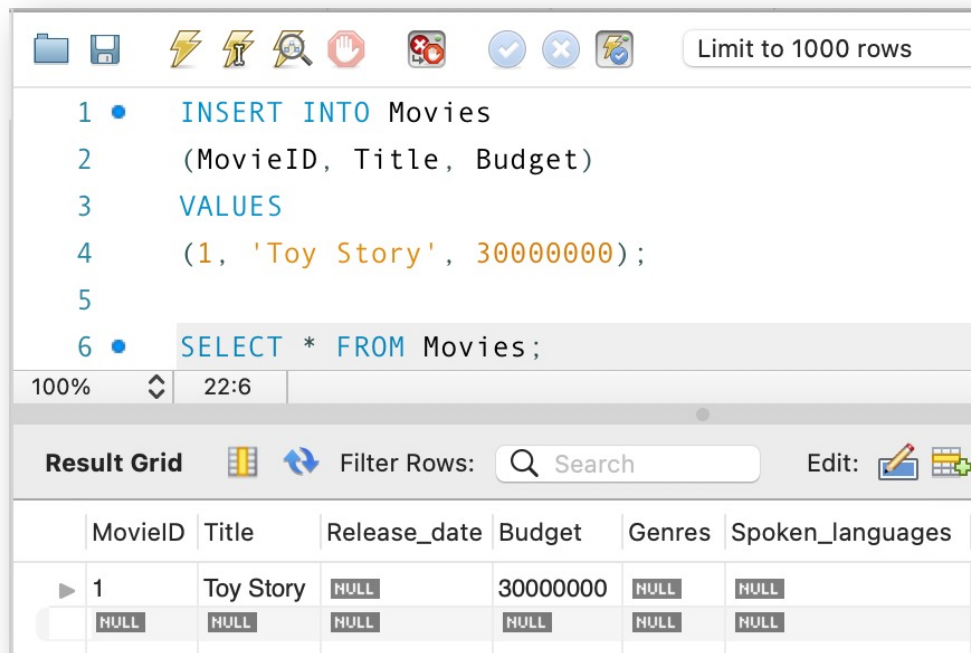
- Once you have defined the Employee table, you can use the following command to insert new tuples:

```
INSERT INTO Employee VALUES  
( 'E22', 'Cole', '7th Street', 'Manhattan', 'A6'),  
( 'E23', 'Smith', 'Reseda Blvd', 'Los Angeles', 'A5'),  
... ;
```

- You can copy a set of values from many tables to another, including transformations, using the command `INSERT INTO ... SELECT ...`

```
INSERT INTO Employee (E#, Name, Street, Place)  
SELECT A#, Name, Street, Place  
FROM Applicants, Adresses  
WHERE AdressFK# = Adress# AND Hired = True
```

Inserting Data with SQL



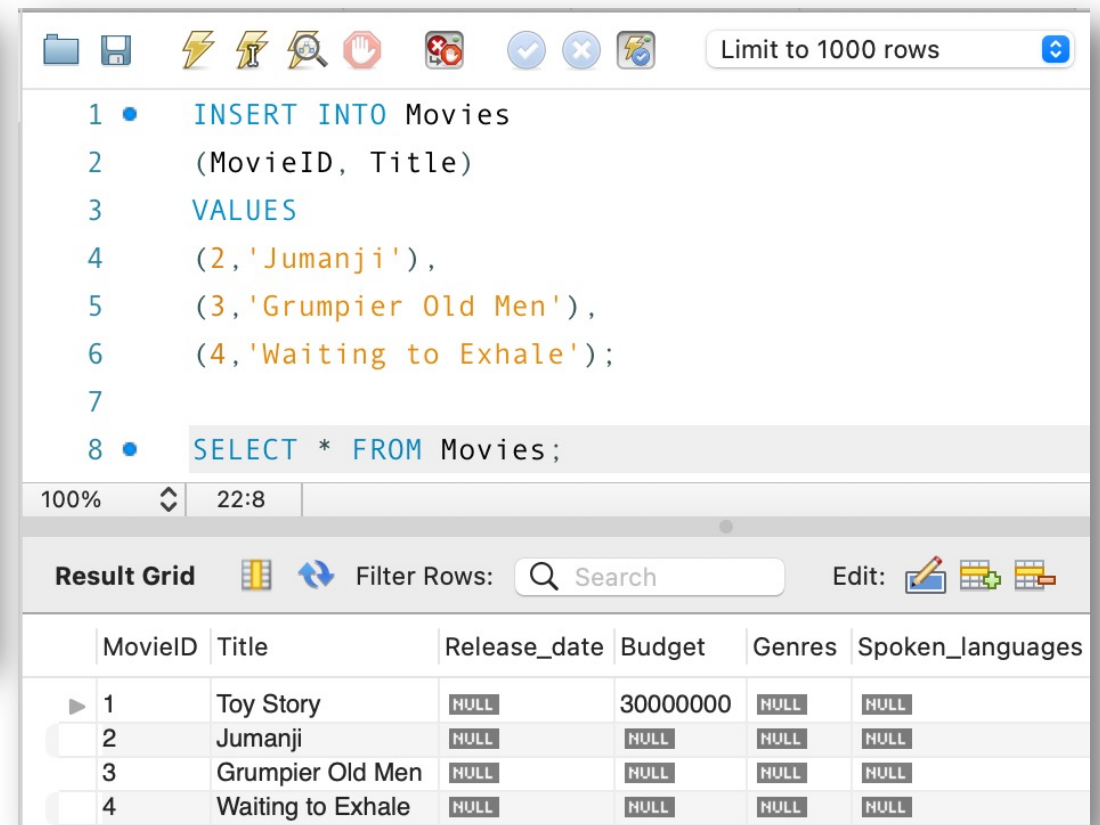
Limit to 1000 rows

```
1 • INSERT INTO Movies
2   (MovieID, Title, Budget)
3   VALUES
4   (1, 'Toy Story', 30000000);
5
6 • SELECT * FROM Movies;
```

100% 22:6

Result Grid Filter Rows: Search Edit:

	MovieID	Title	Release_date	Budget	Genres	Spoken_languages
▶	1	Toy Story	NULL	30000000	NULL	NULL
◀	NULL	NULL	NULL	NULL	NULL	NULL



Limit to 1000 rows

```
1 • INSERT INTO Movies
2   (MovieID, Title)
3   VALUES
4   (2, 'Jumanji'),
5   (3, 'Grumpier Old Men'),
6   (4, 'Waiting to Exhale');
7
8 • SELECT * FROM Movies;
```

100% 22:8

Result Grid Filter Rows: Search Edit:

	MovieID	Title	Release_date	Budget	Genres	Spoken_languages
▶	1	Toy Story	NULL	30000000	NULL	NULL
◀	2	Jumanji	NULL	NULL	NULL	NULL
	3	Grumpier Old Men	NULL	NULL	NULL	NULL
◀	4	Waiting to Exhale	NULL	NULL	NULL	NULL

UPDATE: Changing the data

- You can manipulate the EMPLOYEE table using an UPDATE command:

```
UPDATE    EMPLOYEE
SET       City = 'Basilea'
WHERE     City = 'Basel'
```



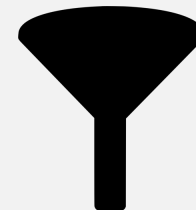
- This example command will replace all instances of residence with the value "Basel" with the new name "Basilea" in all tuples in the EMPLOYEE table.
- The UPDATE change operation is also set-oriented and can change a set of multi-element tuples.
- Be careful when updating productive databases. Always create a backup (e.g. CREATE TABLE ... AS SELECT ...) and test your updates with SELECT statements first (e.g. SELECT ... FROM ... WHERE...)

SELECT: Filter Data

```
SELECT      selected attributes      (Output)
FROM        tables to be searched   (Input)
WHERE       selection condition
            (Processing)
```

- The SELECT clause specifies the list of attributes (columns) in the output
- The FROM clause lists all the required tables as input.
- Filters in the WHERE clause can be described by linking different predicates (=, >, <=, etc) using logical operators such as AND, NOT and OR.

```
SELECT      Name
FROM        Employess
WHERE       City = 'Kent'
```



DELETE: Delete data

- Finally, you can delete entire tables or parts thereof with the DELETE command:

```
DELETE FROM EMPLOYEE  
WHERE City = 'Basilea'
```



- The delete command normally affects a set of tuples if the selection predicate applies to several table entries.
- Be careful when deleting data in productive databases. Always create a backup (e.g. CREATE TABLE ... AS SELECT ...) and test your deletion with SELECT statements first (e.g. SELECT ... FROM ... WHERE...)

Updating and deleting data with SQL

Limit to 1000 rows

```

1 • UPDATE Movies
2   SET Budget = 65000000
3   WHERE MovieID = 2;
4
5 • SELECT * FROM Movies;

```

100% 22:5 1 error found

Result Grid Filter Rows: Search Edit:

MovieID	Title	Release_date	Budget	Genres	Spoken_languages
1	Toy Story	NULL	30000000	NULL	NULL
2	Jumanji	NULL	65000000	NULL	NULL
3	Grumpier Old Men	NULL	NULL	NULL	NULL
4	Waiting to Exhale	NULL	NULL	NULL	NULL

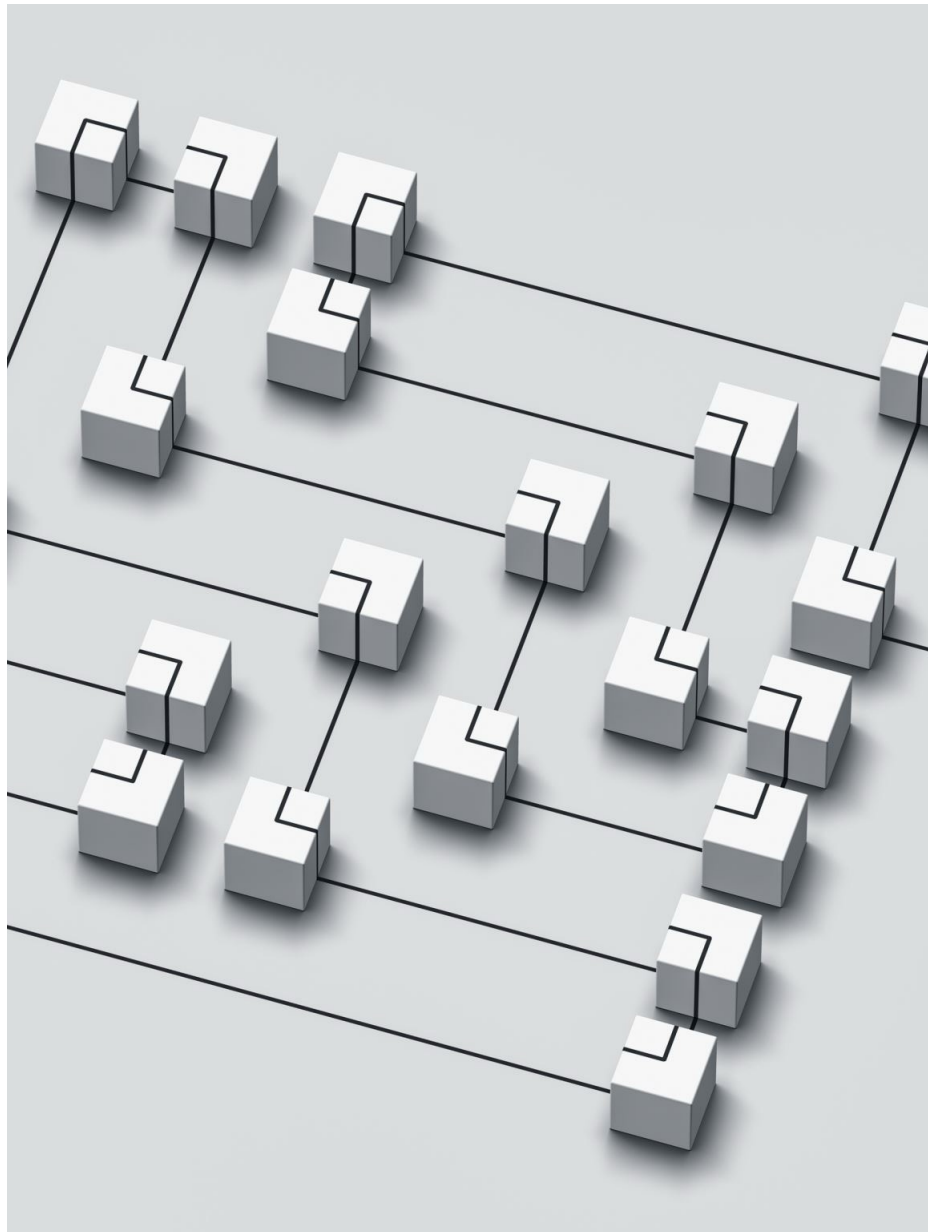
The screenshot shows a database client window. At the top, there's a toolbar with icons for file operations, a search icon, and a refresh icon. Below the toolbar, a text area contains two SQL commands:

```
1 • DELETE FROM Movies WHERE MovieID = 2;
2
3 • SELECT * FROM Movies;
```

Below the text area, there's a status bar showing "100%" zoom and a timer "22:3".

The bottom section is titled "Result Grid" and shows a table with the following columns: MovieID, Title, Release_date, Budget, Genres, and Spoken_languages. The table contains three rows of data:

	MovieID	Title	Release_date	Budget	Genres	Spoken_languages
▶	1	Toy Story	NULL	30000000	NULL	NULL
	3	Grumpier Old Men	NULL	NULL	NULL	NULL
	4	Waiting to Exhale	NULL	NULL	NULL	NULL



2.) Real World Data Sources

A database organizes data records. A database server automates this organization. SQL allows to INSERT data as code.

But where does the data in databases come from in the first place?

What are different **formats** to integrate digital data into an SQL-DB?

Source data often comes in files

The image shows two windows side-by-side. The left window is a PowerShell terminal, and the right window is a CSV editor showing a file named 'movies.csv'.

PowerShell Window:

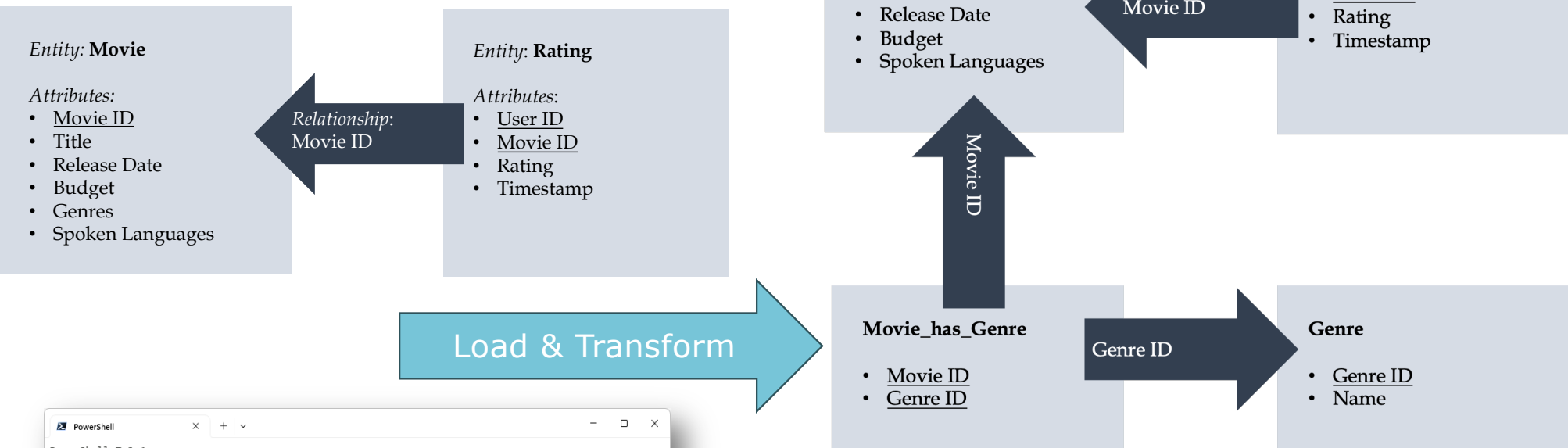
```
PowerShell 7.3.6
PS C:\Users\micha> get-content C:\Users\micha\Downloads\ml-25m\ml-25m\ratings.csv | select -First 15
userId,movieId,rating,timestamp
1,296,5.0,1147880044
1,306,3.5,1147868817
1,307,5.0,1147868828
1,665,5.0,1147878820
1,899,3.5,1147868510
1,1088,4.0,1147868495
1,1175,3.5,1147868826
1,1217,3.5,1147878326
1,1237,5.0,1147868839
1,1250,4.0,1147868414
1,1260,3.5,1147877857
1,1653,4.0,1147868097
1,2011,2.5,1147868079
1,2012,2.5,1147868068
PS C:\Users\micha>
```

movies.csv - Editor Window:

MovieID	Title	Release_date	Budget	Genres	Spoken_languages
"1"	"Toy Story"	"1995-10-30"	"30000000"	"[{"id": 16, "name": "Animation"}, {"id": 35,	
"2"	"Jumanji"	"1995-12-15"	"65000000"	"[{"id": 12, "name": "Adventure"}, {"id": 14,	
"3"	"Grumpier Old Men"	"1995-12-22"	"0"	"[{"id": 10749, "name": "Romance"}, {"id	
"4"	"Waiting to Exhale"	"1995-12-22"	"16000000"	"[{"id": 35, "name": "Comedy"}, {"	
"5"	"Father of the Bride Part II"	"1995-02-10"	"0"	"[{"id": 35, "name": "Comedy"}]"	
"6"	"Heat"	"1995-12-15"	"60000000"	"[{"id": 28, "name": "Action"}, {"id": 80, \	
"7"	"Sabrina"	"1995-12-15"	"58000000"	"[{"id": 35, "name": "Comedy"}, {"id": 10749,	
"8"	"Tom and Huck"	"1995-12-22"	"0"	"[{"id": 28, "name": "Action"}, {"id": 12, \	
"9"	"Sudden Death"	"1995-12-22"	"35000000"	"[{"id": 28, "name": "Action"}, {"id":	
"10"	"GoldenEye"	"1995-11-16"	"58000000"	"[{"id": 12, "name": "Adventure"}, {"id": 28,	
"11"	"The American President"	"1995-11-17"	"62000000"	"[{"id": 35, "name": "Comedy\"	
"12"	"Dracula: Dead and Loving It"	"1995-12-22"	"0"	"[{"id": 35, "name": "Comedy"}, {"	
"13"	"Balto"	"1995-12-22"	"0"	"[{"id": 10751, "name": "Family"}, {"id": 16, \nam	
"14"	"Nixon"	"1995-12-22"	"44000000"	"[{"id": 36, "name": "History"}, {"id": 18, \	
"15"	"Cutthroat Island"	"1995-12-22"	"98000000"	"[{"id": 28, "name": "Action"}, {"	
"16"	"Casino"	"1995-11-22"	"52000000"	"[{"id": 18, "name": "Drama"}, {"id": 80, \n	
"17"	"Sense and Sensibility"	"1995-12-13"	"16500000"	"[{"id": 18, "name": "Drama"}, {"i	

At the bottom of the CSV editor, it shows 'Ze 6, Sp 31', '100%', 'Unix (LF)', and 'UTF-8'.

Loading Files into the Target Structure



Load & Transform

```
PowerShell 7.3.6
PS C:\Users\micha> get-content C:\Users\micha\Downloads\ml-25m\ml-25m\ratings.csv | select -First 15
userId,movieId,rating,timestamp
1,296,5.0,1147880044
1,306,3.5,1147868817
1,307,5.0,1147868828
1,665,5.0,1147878820
1,899,3.5,1147868510
1,1088,4.0,1147868495
1,1175,3.5,1147868826
1,1217,3.5,1147878326
1,1237,5.0,1147868839
1,1250,4.0,1147868414
1,1260,3.5,1147877857
1,1653,4.0,1147868097
1,2011,2.5,1147868079
1,2012,2.5,1147868068
PS C:\Users\micha>
```

```
movies.csv - Editor
Datei Bearbeiten Ansicht
"MovieID" "Title" "Release_date" "Budget" "Genres" "Spoken_languages"
"1" "Toy Story" "1995-10-30" "30000000" "[{"id": 16, "name": "Animation"}, {"id": 35, "name": "Adventure"}, {"id": 14, "name": "Comedy"}, {"id": 10749, "name": "Romance"}]"
"2" "Jumanji" "1995-12-15" "65000000" [{"id": 12, "name": "Adventure"}, {"id": 14, "name": "Comedy"}, {"id": 10749, "name": "Romance"}]"
"3" "Grumpier Old Men" "1995-12-22" "0" [{"id": 10749, "name": "Romance"}, {"id": 14, "name": "Comedy"}, {"id": 10749, "name": "Romance"}]"
"4" "Waiting to Exhale" "1995-12-22" "160000000" [{"id": 35, "name": "Comedy"}, {"id": 14, "name": "Comedy"}, {"id": 10749, "name": "Romance"}]"
"5" "Father of the Bride Part II" "1995-02-10" "0" [{"id": 35, "name": "Comedy"}, {"id": 14, "name": "Comedy"}, {"id": 10749, "name": "Romance"}]"
"6" "Heat" "1995-12-15" "60000000" [{"id": 28, "name": "Action"}, {"id": 80, "name": "Drama"}, {"id": 10749, "name": "Romance"}]"
"7" "Sabrina" "1995-12-15" "58000000" [{"id": 35, "name": "Comedy"}, {"id": 14, "name": "Comedy"}, {"id": 10749, "name": "Romance"}]"
"8" "Tom and Huck" "1995-12-22" "0" [{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 10749, "name": "Romance"}]"
"9" "Sudden Death" "1995-12-22" "35000000" [{"id": 28, "name": "Action"}, {"id": 14, "name": "Comedy"}, {"id": 10749, "name": "Romance"}]"
"10" "GoldenEye" "1995-11-16" "58000000" [{"id": 12, "name": "Adventure"}, {"id": 28, "name": "Action"}, {"id": 10749, "name": "Romance"}]"
"11" "The American President" "1995-11-17" "62000000" [{"id": 35, "name": "Comedy"}, {"id": 14, "name": "Comedy"}, {"id": 10749, "name": "Romance"}]"
"12" "Dracula: Dead and Loving It" "1995-12-22" "0" [{"id": 35, "name": "Comedy"}, {"id": 14, "name": "Comedy"}, {"id": 10749, "name": "Romance"}]"
"13" "Balto" "1995-12-22" "0" [{"id": 10751, "name": "Family"}, {"id": 16, "name": "Animation"}, {"id": 14, "name": "Comedy"}, {"id": 10749, "name": "Romance"}]"
"14" "Nixon" "1995-12-22" "44000000" [{"id": 36, "name": "History"}, {"id": 18, "name": "Drama"}, {"id": 10749, "name": "Romance"}]"
"15" "Cutthroat Island" "1995-12-22" "98000000" [{"id": 28, "name": "Action"}, {"id": 14, "name": "Comedy"}, {"id": 10749, "name": "Romance"}]"
"16" "Casino" "1995-11-22" "52000000" [{"id": 18, "name": "Drama"}, {"id": 80, "name": "Drama"}, {"id": 10749, "name": "Romance"}]"
"17" "Sense and Sensibility" "1995-12-13" "16500000" [{"id": 18, "name": "Drama"}, {"id": 14, "name": "Comedy"}, {"id": 10749, "name": "Romance"}]"
Ze 6, Sp 31 100% Unix (LF) UTF-8
```

HSLU

Normalized DB-Schema

MySQL Server

Loading files with SQL INSERT

Inserting data with SQL DML is possible, as we have seen (INSERT). However, in our example we want to insert a whole file, e.g. "movies.csv".

- Is loading entire files using the SQL INSERT statement possible in principle?
- If no, why not?
- If yes, how exactly could we accomplish this?

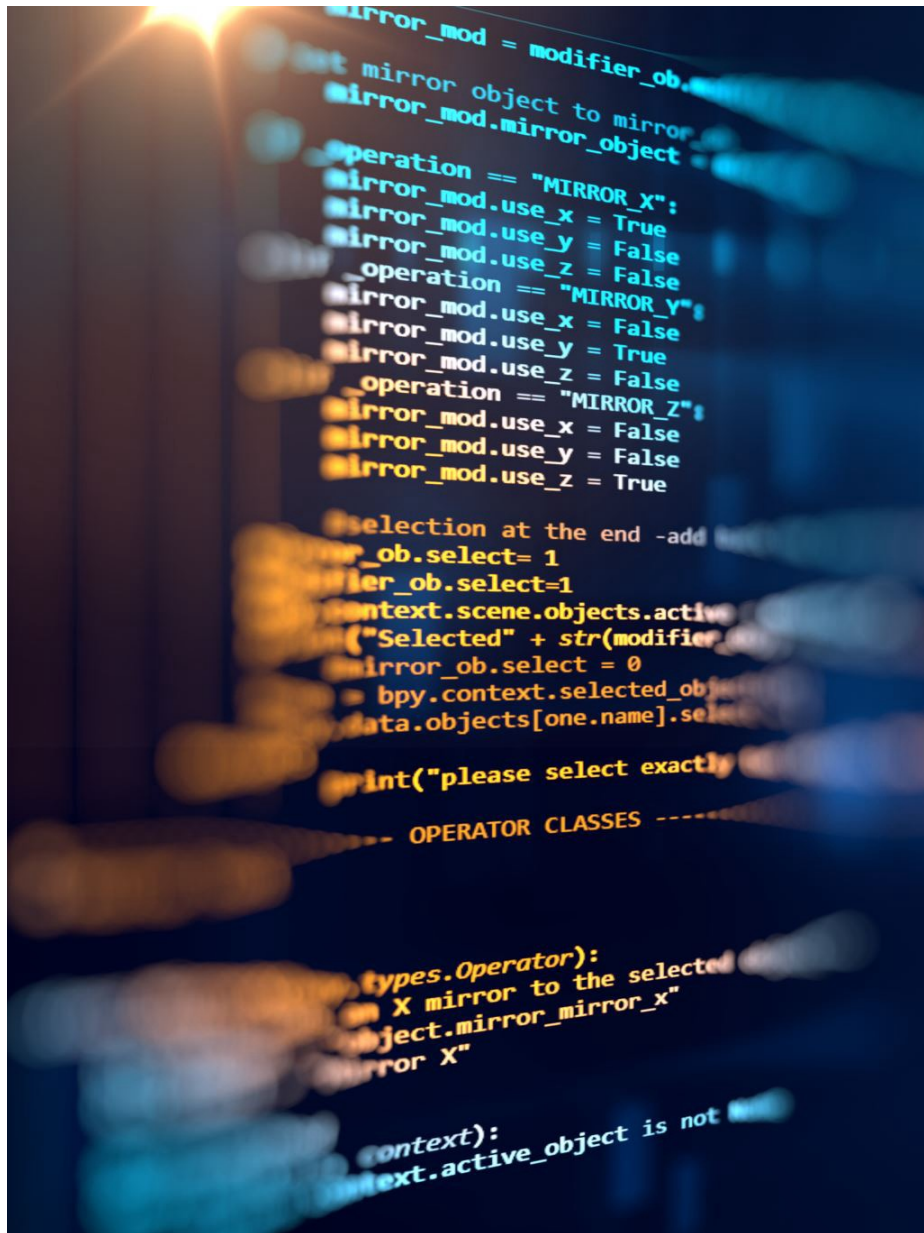
Hint: Use NoSQL Bulk Inserts instead of SQL INSERT command for loading large CSV files

SQL INSERT scripts are not very efficient, because SQL needs to be parsed. Also, they can't load files directly.

ETL tools (e.g. Tableau prep) and APIs (e.g. Python Pandas) can help

Most importantly, DBMS-specific, **native loading utilities** are much faster!

- MySQL Workbench: very slow, not usable (translates to SQL INSERT)
- MySQL **LOAD DATA LOCAL INFILE** : CSV load faster than INSERT
- MySQL Shell: **util.importTable**. Multithreading, parallel loading (parallel LOAD)
- MS SQL: BULK INSERT
- PostgreSQL: COPY



3.) ETL

Extract Transform Load

Information Technology

October 7, 2025

FH Zentralschweiz

Extract Transform Load (ETL)



ETL Example (MySQL): Type Conversion on load using the LOAD command

How can we make this loading process efficient for large data volumes?

```
LOAD DATA LOCAL INFILE '/path/to/ratings.csv'  
INTO TABLE Ratings  
FIELDS TERMINATED BY ','  
ENCLOSED BY '"'  
IGNORE 1 LINES  
(UserID, MovieID, Rating, @Timestamp)  
SET Timestamp = FROM_UNIXTIME(@Timestamp);
```

Transform before loading

Example: Result for movies.csv

Limit to 1000 rows

```

1 • SET GLOBAL local_infile = true;
2
3 • USE RecommenDB;
4
5 • LOAD DATA LOCAL INFILE 'C:/Users/micha/movies.csv'
6   INTO TABLE Movies
7   FIELDS ENCLOSED BY '"' ESCAPED BY '\\'
8   IGNORE 1 LINES;
9
10 • SELECT * FROM Movies;
  
```

Result Grid

	MovieID	Title	Release_date	Budget	Genres	Spoken_lang
▶	1	Toy Story	1995-10-30	30000000	[{"id": 16, "name": "Animation"}, {"id": 35, "name": "Adventure"}]	[{"name": "English"}]
	2	Jumanji	1995-12-15	65000000	[{"id": 12, "name": "Adventure"}, {"id": 14, "name": "Comedy"}]	[{"name": "English"}]
	3	Grumpier Old Men	1995-12-22	0	[{"id": 10749, "name": "Romance"}, {"id": 35, "name": "Comedy"}]	[{"name": "English"}]
	4	Waiting to Exhale	1995-12-22	16000000	[{"id": 35, "name": "Comedy"}, {"id": 18, "name": "Drama"}]	[{"name": "English"}]
	5	Father of the Bride Part II	1995-02-10	0	[{"id": 35, "name": "Comedy"}]	[{"name": "English"}]

Movies 3 × Apply Revert

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓	1 22:22:20	SET GLOBAL local_infile = true	0 row(s) affected	0.000 sec
✓	2 22:22:22	USE RecommenDB	0 row(s) affected	0.000 sec
⚠	3 22:22:23	LOAD DATA LOCAL INFILE 'C:/Users/micha/movies.csv'	45463 row(s) affected, 62 warning(s): 1062 D...	2.421 sec
✓	4 22:22:28	SELECT * FROM Movies LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.016 sec

Example: Result for ratings.csv

11 • **SELECT * FROM Ratings;**

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows

	UserID	MovieID	Rating	Timestamp
▶	8	1	4	2002-02-11 17:06:36
	9	1	5	2004-01-11 17:06:20
	12	1	4	1999-11-29 22:50:05
	20	1	4	2013-05-12 14:22:28
	24	1	4	2001-01-19 03:05:38

Ratings 2 x Apply Revert

Output

Action Output

	#	Time	Action	Message	Duration / Fetch
✓	1	11:12:28	SET foreign_key_checks = 0	0 row(s) affected	0.000 sec
✓	2	11:12:30	LOAD DATA LOCAL INFILE 'C:\\User...	26024289 row(s) affected Record...	2088.422 sec
✓	3	15:46:28	SELECT * FROM Ratings LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec

Loading large Datasets: the Table Import Utility in the MySQL Shell

```
micha — Mysqlsh — 80x24
Server version: 8.0.21 MySQL Community Server - GPL
No default schema selected; type \use <schema> to set one.
MySQL localhost:3306 ssl JS > \sql SET GLOBAL local_infile = true; #allow local files
Query OK, 0 rows affected (0.0002 sec)
MySQL localhost:3306 ssl JS > \sql SET GLOBAL connect_timeout=50000; #longer runtime
Query OK, 0 rows affected (0.0002 sec)
MySQL localhost:3306 ssl JS > util.importTable("/Users/micha/Downloads/archive/ratings.csv", {schema:"themoviesdb", table:"ratings", dialect:"csv", skipRows:1, showProgress:true, fieldsOptionallyEnclosed:true, fieldsTerminatedBy:",", linesTerminatedBy:"\n", fieldsEnclosedBy:'"',})
Importing from file '/Users/micha/Downloads/archive/ratings.csv' to table `themoviesdb`.`ratings` in MySQL Server at localhost:3306 using 8 threads
0% (2.10 MB / 709.55 MB), 0.00 B/s
[Worker005] ratings.csv: Records: 1805960 Deleted: 0 Skipped: 0 Warnings: 0
[Worker000] ratings.csv: Records: 1816917 Deleted: 0 Skipped: 0 Warnings: 0
[Worker001] ratings.csv: Records: 1808108 Deleted: 0 Skipped: 0 Warnings: 0
[Worker003] ratings.csv: Records: 1874598 Deleted: 0 Skipped: 0 Warnings: 0
[Worker002] ratings.csv: Records: 1871838 Deleted: 0 Skipped: 0 Warnings: 0
[Worker007] ratings.csv: Records: 1874491 Deleted: 0 Skipped: 0 Warnings: 0
[Worker006] ratings.csv: Records: 1874063 Deleted: 0 Skipped: 0 Warnings: 0
[Worker004] ratings.csv: Records: 1914954 Deleted: 0 Skipped: 0 Warnings: 0
63% (450.33 MB / 709.55 MB), 4.01 MB/s
```

Parallel Loads with independend commit points

How does that
help loading very
large datasets?

MySQL Workbench

root@localhost

Administration Schemas Shell professors Administration - Client Connections

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Client Connections

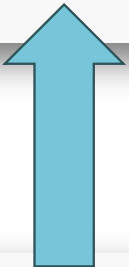
Threads Connected: 13 Threads Running: 10 Threads Created: 23 Threads Cached: 0 Rejected (over limit): 0
Total Connections: 88 Connection Limit: 151 Aborted Clients: 12 Aborted Connections: 0 Errors: 0

Id	User	Host	DB	Command	Time	State	Thread...	Type	Name	Parent...	Instrumented	Info
5	event_sch...	None	None	Sleep	0	Waiting on...	48	FOREGRO...	thread/sql/...	1	YES	NULL
7	None	None	None	Daemon	197824	Suspending	51	FOREGRO...	thread/sql/...	1	YES	NULL
59	root	localhost	themoviesdb	Sleep	178	None	103	FOREGRO...	thread/sql/...	0	YES	KILL QUERY 60
60	root	localhost	themoviesdb	Sleep	109	None	104	FOREGRO...	thread/sql/...	0	YES	create table ratings (...)
78	root	localhost	None	Query	0	executing	130	FOREGRO...	thread/sql/...	1	YES	SELECT t.PROCESSLIST_ID,IF...
79	root	localhost	None	Sleep	1	None	131	FOREGRO...	thread/sql/...	1	YES	SHOW GLOBAL STATUS
80	root	localhost	None	Sleep	61	None	132	FOREGRO...	thread/sql/...	0	YES	SHOW GLOBAL VARIABLES LI...
81	root	localhost	themoviesdb	Query	61	executing	133	FOREGRO...	thread/sql/...	0	YES	LOAD DATA LOCAL INFILE '/Us...
82	root	localhost	themoviesdb	Query	61	executing	134	FOREGRO...	thread/sql/...	0	YES	LOAD DATA LOCAL INFILE '/Us...
83	root	localhost	themoviesdb	Query	61	executing	135	FOREGRO...	thread/sql/...	0	YES	LOAD DATA LOCAL INFILE '/Us...
84	root	localhost	themoviesdb	Query	61	executing	136	FOREGRO...	thread/sql/...	0	YES	LOAD DATA LOCAL INFILE '/Us...
85	root	localhost	themoviesdb	Query	61	executing	137	FOREGRO...	thread/sql/...	0	YES	LOAD DATA LOCAL INFILE '/Us...
86	root	localhost	themoviesdb	Query	61	executing	138	FOREGRO...	thread/sql/...	0	YES	LOAD DATA LOCAL INFILE '/Us...
87	root	localhost	themoviesdb	Query	61	executing	139	FOREGRO...	thread/sql/...	0	YES	LOAD DATA LOCAL INFILE '/Us...
88	root	localhost	themoviesdb	Query	61	executing	140	FOREGRO...	thread/sql/...	0	YES	LOAD DATA LOCAL INFILE '/Us...

Loading XML Data into MySQL

```
USE test;

CREATE TABLE person (
  person_id INT NOT NULL PRIMARY KEY,
  fname VARCHAR(40) NULL,
  lname VARCHAR(40) NULL,
  created TIMESTAMP
);
```



```
<list>
  <person person_id="1" fname="Kapek" lname="Sainnouine"/>
  <person person_id="2" fname="Sajon" lname="Rondela"/>
  <person person_id="3"><fname>Likame</fname><lname>Örrtmons</lname></person>
  <person person_id="4"><fname>Slar</fname><lname>Manlanth</lname></person>
  <person><field name="person_id">5</field><field name="fname">Stoma</field>
    <field name="lname">Milu</field></person>
  <person><field name="person_id">6</field><field name="fname">Nirtam</field>
    <field name="lname">Sklöd</field></person>
  <person person_id="7"><fname>Sungam</fname><lname>Dulbåd</lname></person>
  <person person_id="8" fname="Sraref" lname="Encmelt"/>
</list>
```



```
mysql> LOAD XML LOCAL INFILE 'person.xml'
-> INTO TABLE person
-> ROWS IDENTIFIED BY '<person>';
```

```
Query OK, 8 rows affected (0.00 sec)
Records: 8 Deleted: 0 Skipped: 0 Warnings: 0
```

<https://dev.mysql.com/doc/refman/8.0/en/load-xml.html>

Loading Json Data into MySQL: util.importJson

```
MySQL localhost:33300+ JS util.importJson("/path_to_file/zips.json",
{schema: "test", collection: "zips_collection"});
Importing from file "/path_to_file/zips.json" to collection `test`.`zips_collect
ion` in MySQL Server at localhost:33300

.. 29353.. 29353
Processed 3.15 MB in 29353 documents in 1.3716 sec (21.40K documents/s)
Total successfully imported documents 29353 (21.40K documents/s)

MySQL localhost:33300+ JS util.importJson("/path_to_file/zips.json",
{schema: "test", table: "zips_table"});
Importing from file "/path_to_file/zips.json" to table `test`.`zips_table` in My
SQL Server at localhost:33300

.. 29353.. 29353
Processed 3.15 MB in 29353 documents in 1.2969 sec (22.63K documents/s)
Total successfully imported documents 29353 (22.63K documents/s)
```

<https://dev.mysql.com/blog-archive/import-json-to-mysql-made-easy-with-the-mysql-shell/>

HSLU
<https://dev.mysql.com/doc/mysql-shell/8.3/en/mysql-shell-utilities-json.html>



Distributed Data

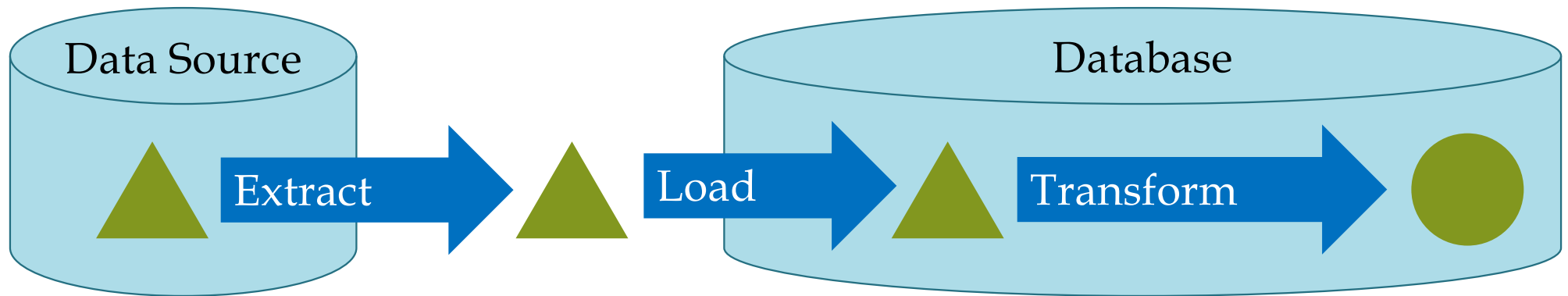
Let's assume that the data we need for a database application is distributed across many databases.

What are different ways to combine the distributed data in order to evaluate it?

4.) EL-T

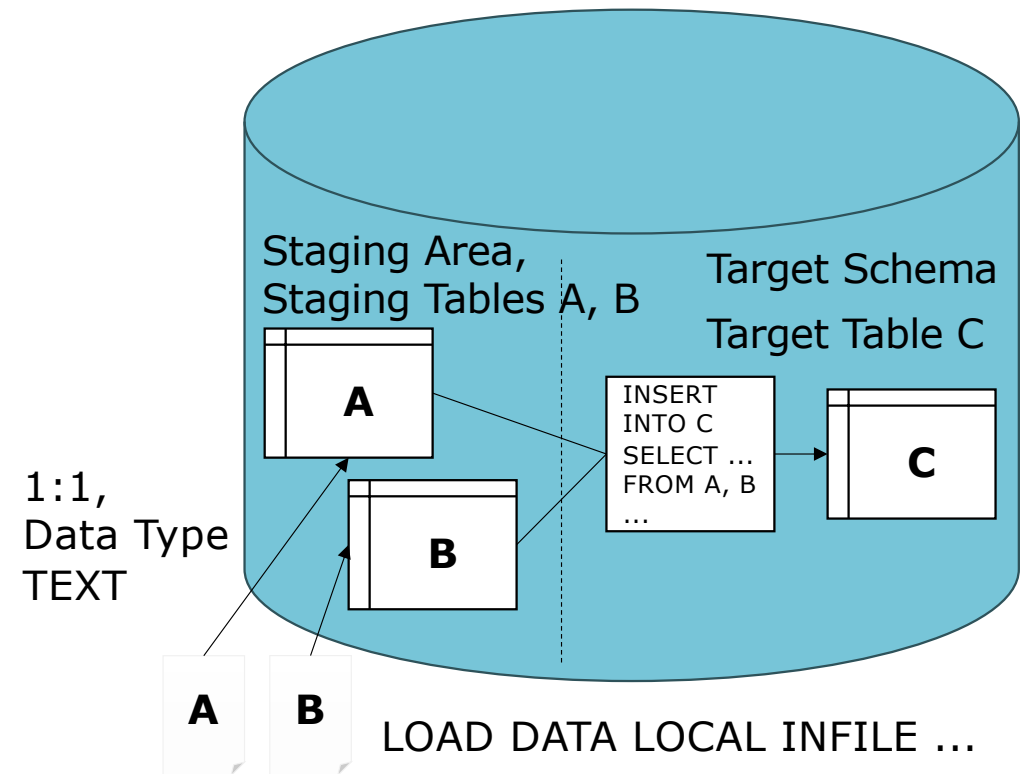
Extract => Load => Transform

Extract Load Transform (ELT)



Extract, Load, Transform by Integrating Data using SQL: INSERT INTO ... SELECT

- How can you fit data from different source files to one target table?
- Extract, Load, Transform (ELT) can combine data from several source files.
- You LOAD the files into tables **corresponding 1:1 to the the source format**, where all columns are of data Type **TEXT**. These are called **Staging Tables**.
- Then you INSERT into the target table the data set defined by a SELECT query over the source tables in one **INSERT INTO ... SELECT** command.
- Thus you can slice and dice the data according to your needs.



How to transform the JSON Genres of Movies

```
[ {"id": 16, "name": "Animation"},  
  {"id": 35, "name": "Comedy"},  
  {"id": 10751, "name": "Family"} ]
```

Genres for movie 1

movies.csv - Editor						
Datei Bearbeiten Ansicht						
"MovieID"	"Title"	"Release_date"	"Budget"	"Genres"	"Spoken_languages"	
"1"	"Toy Story"	"1995-10-30"	"30000000"	"[{\"id\": 16, \"name\": \"Animation\"}, {\"id\": 35,		
"2"	"Jumanji"	"1995-12-15"	"65000000"	"[{\"id\": 12, \"name\": \"Adventure\"}, {\"id\": 14,		
"3"	"Grumpier Old Men"	"1995-12-22"	"0"	"[{\"id\": 10749, \"name\": \"Romance\"}, {\"id		
"4"	"Waiting to Exhale"	"1995-12-22"	"16000000"	"[{\"id\": 35, \"name\": \"Comedy\"}, {\"		
"5"	"Father of the Bride Part II"	"1995-02-10"	"0"	"[{\"id\": 35, \"name\": \"Comedy\"}]"		
"6"	"Heat"	"1995-12-15"	"60000000"	"[{\"id\": 28, \"name\": \"Action\"}, {\"id\": 80, \"		
"7"	"Sabrina"	"1995-12-15"	"58000000"	"[{\"id\": 35, \"name\": \"Comedy\"}, {\"id\": 10749,		

Example (SQL): Normalisation of JSON Genres

General rule for ELT:

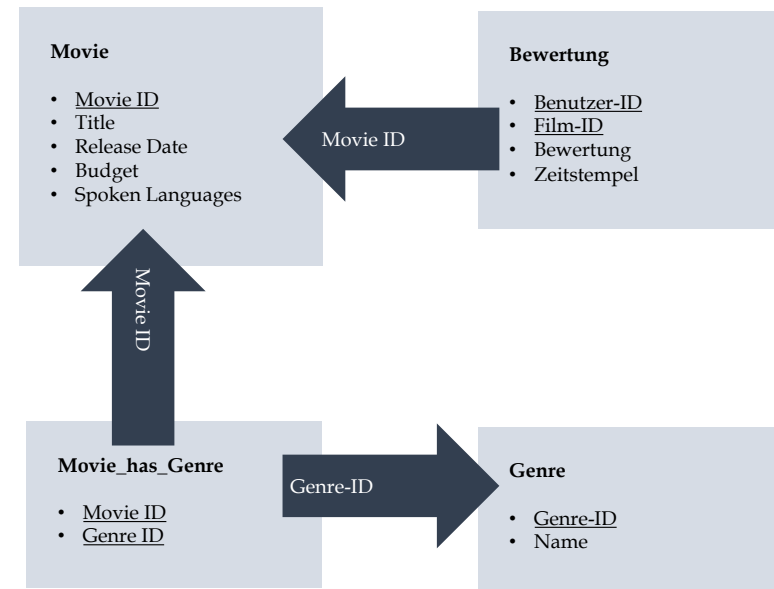
```
INSERT INTO <destination>
SELECT <transformation>
```

Example: Genres for movie #1

```
[ {"id": 16, "name": "Animation"},
  {"id": 35, "name": "Comedy"},
  {"id": 10751, "name": "Family"} ]
```

Extract from Staging table and Load into the target tables:

```
INSERT INTO Genre
SELECT DISTINCT GenreID, GenreName
FROM Movies
JOIN JSON_TABLE (
  Genres, '$[*]' COLUMNS (
    GenreID INTEGER PATH '$.id',
    GenreName TEXT PATH '$.name' )
) AS Genres;
```



```
INSERT INTO Movie_has_Genre
SELECT DISTINCT MovieID, GenreID
FROM Movies
JOIN JSON_TABLE ( Genres, '$[*]'
COLUMNS ( GenreID INTEGER PATH '$.id' )
) AS Genres;
```

Result for Genres

```
3
4  INSERT INTO Genres
5    SELECT DISTINCT GenreID, GenreName
6    FROM Movies
7  JOIN JSON_TABLE (
8    Genres, '$[*]' COLUMNS (
9      GenreID INTEGER PATH '$.id',
10     GenreName TEXT PATH '$.name' )
11  ) AS Genres;
12
13  SELECT * FROM Genres;
14
```

100% 12:13

Result Grid Filter Rows: Search

GenreID	Name
16	Animation
18	Drama
27	Horror
28	Action
35	Comedy
36	History
37	Western
53	Thriller

Result for Movie_has_Genre

```
12
13 SELECT * FROM Genres;
14
15 INSERT INTO Movie_has_Genre
16   SELECT DISTINCT MovieID, GenreID
17   FROM Movies
18   JOIN JSON_TABLE ( Genres, '$[*]'
19     COLUMNS ( GenreID INTEGER PATH '$.id' )
20   ) AS Genres;
21
22 SELECT * FROM Movie_has_Genre;
```

100% 18:22

Result Grid Filter Rows: Search Edit

	MovieID	GenreID
	1	16
	1	35
	1	10751
	2	12
	2	14
	2	10751
	3	35
	3	10749
	4	10



Data Matching

☞ What if the same content is structured differently in different databases that we want to integrate?

- ☞ E.g. corporate fusion like CS and UBS. How to match customer databases without matching ID?
- ☞ E.g. the same customer number is called in 2 DBs once C_ID and once C_NR.

☞ Or: what happens if the same structures store different contents?

- ☞ E.g. both DBs have column C_ID, but the numbers do not match.

☞ What are the possible solutions?

